The SPIN Model Checker

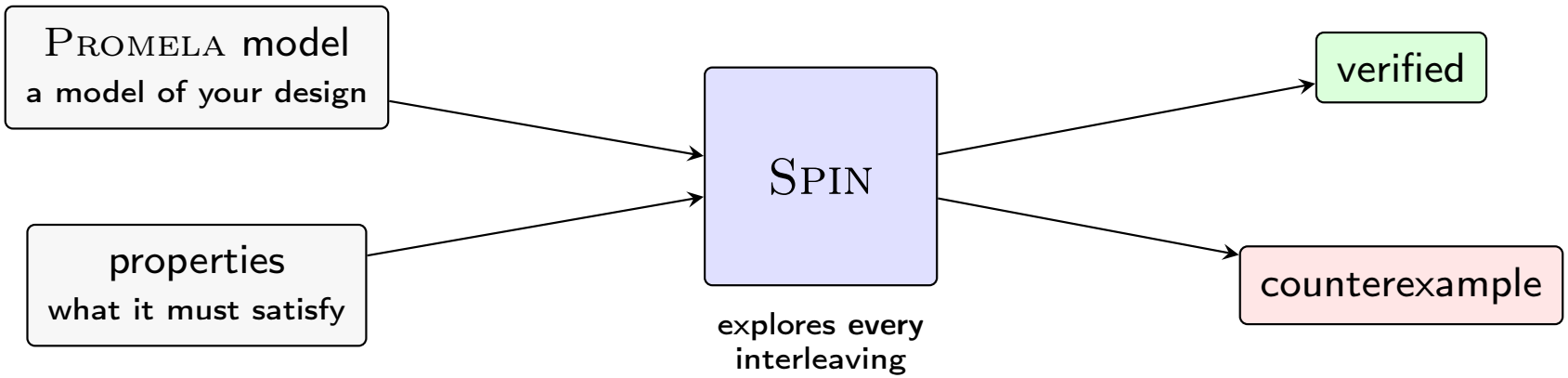
Verifying Concurrent and Distributed Software

Klaus Havelund
Jet Propulsion Laboratory
California Institute of Technology

JPL — September 2026

What this talk is about

SPIN is a model checker for *concurrent* and *distributed* software.

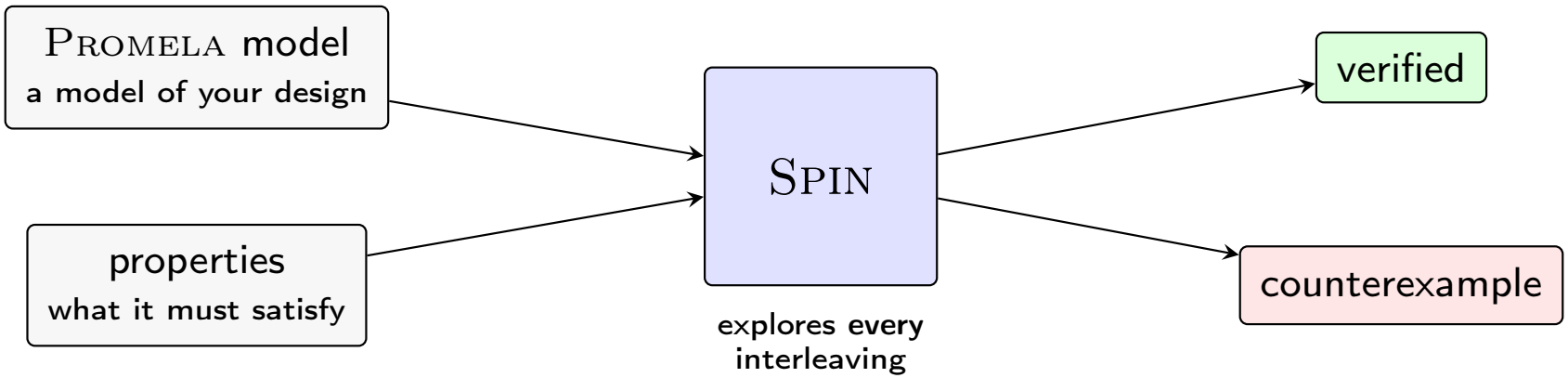


Either it finds no violation in *any* interleaving, or it hands back the one execution that breaks the property.

- Developed by **Gerard Holzmann**, originally at Bell Labs, later at JPL.
- ACM **Software System Award**, 2001.
- Open source, actively used in industry and at NASA for 30+ years.
- `https://spinroot.com`

What this talk is about

SPIN is a model checker for *concurrent* and *distributed* software.



Either it finds no violation in *any* interleaving, or it hands back the one execution that breaks the property.

- Developed by **Gerard Holzmann**, originally at Bell Labs, later at JPL.
- ACM **Software System Award**, 2001.
- Open source, actively used in industry and at NASA for 30+ years.
- `https://spinroot.com`

Motivation: why concurrency defeats testing

Six lines of obviously correct code

Two instruments share a 10 A power bus. Each draws 6 A, and each asks the power manager before switching on.

```
byte avail   = 10;              /* SHARED: amps left on the bus  */
byte powered = 0;               /* SHARED: instruments now on    */

inline request_power(amps) {
  if
  :: avail >= amps →            /* CHECK: is there enough?       */
       avail = avail - amps;    /* ACT:   reserve it             */
       powered++                /*        switch on              */
  :: else → skip                /* denied - stay off             */
  fi
}

active proctype InstrumentA() { request_power(6) }   /* thread 1 */
active proctype InstrumentB() { request_power(6) }   /* thread 2 */
```

The interleaving that flies

step	Instrument A	Instrument B	avail	powered
0	*(initial)*		10	0
1	`avail >= 6 ?  yes`		10	0
2		`avail >= 6 ?  yes`	10	0
3	`avail = 10 - 6`		4	0
4	`powered++`		4	1
5		`avail = 4 - 6`	**254**	1
6		`powered++`	**254**	**2**

Two failures for the price of one

- 12 **A drawn from a** 10 **A bus.** Brownout, undervoltage trip, or a reset — on hardware, not in the model.
- `avail` **underflowed** $4 - 6 = -2$, and as a `byte` that is 254. The power manager now believes it has 254 A to give away. *Every subsequent request succeeds.*

The second failure is the dangerous one: the system does not crash here. It quietly stops enforcing the budget, and fails somewhere else, much later.

The interleaving that flies

step	Instrument A	Instrument B	avail	powered
0	*(initial)*		10	0
1	`avail >= 6 ?  yes`		10	0
2		`avail >= 6 ?  yes`	10	0
3	`avail = 10 - 6`		4	0
4	`powered++`		4	1
5		`avail = 4 - 6`	**254**	1
6		`powered++`	**254**	**2**

Two failures for the price of one

- 12 **A drawn from a** 10 **A bus.** Brownout, undervoltage trip, or a reset — on hardware, not in the model.
- `avail` **underflowed** $4 - 6 = -2$, and as a `byte` that is 254. The power manager now believes it has 254 A to give away. *Every subsequent request succeeds.*

The second failure is the dangerous one: the system does not crash here. It quietly stops enforcing the budget, and fails somewhere else, much later.

The interleaving that flies

step	Instrument A	Instrument B	avail	powered
0	*(initial)*		10	0
1	`avail >= 6 ?  yes`		10	0
2		`avail >= 6 ?  yes`	10	0
3	`avail = 10 - 6`		4	0
4	`powered++`		4	1
5		`avail = 4 - 6`	**254**	1
6		`powered++`	**254**	**2**

Two failures for the price of one

- 12 **A drawn from a** 10 **A bus.** Brownout, undervoltage trip, or a reset — on hardware, not in the model.
- `avail` **underflowed** $4 - 6 = -2$, and as a `byte` that is 254. The power manager now believes it has 254 A to give away. *Every subsequent request succeeds.*

The second failure is the dangerous one: the system does not crash here. It quietly stops enforcing the budget, and fails somewhere else, much later.

Why testing does not find this

The window is **two instructions wide** — it needs a preemption landing exactly between the check and the act.

The defect is in the **interleaving**. Reviewing interleavings is far harder than reviewing sequential behavior.

Why testing does not find this

The window is **two instructions wide** — it needs a preemption landing exactly between the check and the act.

The defect is in the **interleaving**. Reviewing interleavings is far harder than reviewing sequential behavior.

The number of interleavings explodes

n tasks, each with m atomic steps, gives

$$\frac{(n \cdot m)!}{(m!)^n}$$

distinct interleavings.

tasks	steps each	interleavings
2	2	6
3	3	1 680
4	4	63 063 000
5	5	$\approx 6.2 \times 10^{14}$
6	6	$\approx 2.7 \times 10^{24}$

Six tasks of six steps have more interleavings than there are **stars in the observable universe** ($\approx 2 \times 10^{23}$).

This is the famous **state explosion problem**, and it is the central obstacle for model checking. We return to it later.

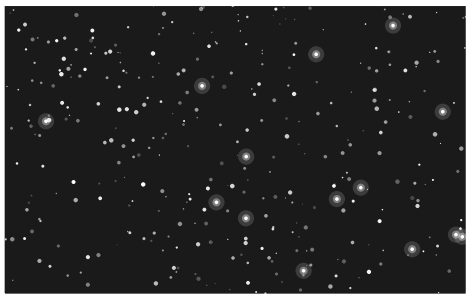
The number of interleavings explodes

n tasks, each with m atomic steps, gives

$$\frac{(n \cdot m)!}{(m!)^n}$$

distinct interleavings.

tasks	steps each	interleavings
2	2	6
3	3	1 680
4	4	63 063 000
5	5	$\approx 6.2 \times 10^{14}$
6	6	$\approx 2.7 \times 10^{24}$



Six tasks of six steps have more interleavings than there are **stars in the observable universe** ($\approx 2 \times 10^{23}$).

This is the famous **state explosion problem**, and it is the central obstacle for model checking. We return to it later.

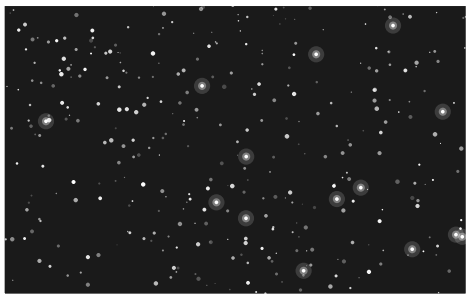
The number of interleavings explodes

n tasks, each with m atomic steps, gives

$$\frac{(n \cdot m)!}{(m!)^n}$$

distinct interleavings.

tasks	steps each	interleavings
2	2	6
3	3	1 680
4	4	63 063 000
5	5	$\approx 6.2 \times 10^{14}$
6	6	$\approx 2.7 \times 10^{24}$



Six tasks of six steps have more interleavings than there are **stars in the observable universe** ($\approx 2 \times 10^{23}$).

This is the famous **state explosion problem**, and it is the central obstacle for model checking. We return to it later.

The idea of model checking

A model checker takes a **finite-state model** M and a **property** φ, and decides whether

$$M \models \varphi$$

If not, it returns a **counterexample**: a concrete execution that violates φ.

	Testing	**Model checking**
What runs	The real system	A model of the system
Coverage	One execution per test	All executions
Guarantee	Finds bugs, cannot prove absence	Proves absence, within the model
Scale	Full systems	Limited by the state space
Effort	No model needed	A model must be built

The idea of model checking

A model checker takes a **finite-state model** M and a **property** φ, and decides whether

$$M \models \varphi$$

If not, it returns a **counterexample**: a concrete execution that violates φ.

	Testing	**Model checking**
What runs	The real system	A model of the system
Coverage	One execution per test	All executions
Guarantee	Finds bugs, cannot prove absence	Proves absence, within the model
Scale	Full systems	Limited by the state space
Effort	No model needed	A model must be built

PROMELA: **the modeling language**

PROMELA in one slide

Process Meta Language. C-like syntax, but it is a **modeling** language, not a programming language.

- **Processes** (`proctype`) — the units of concurrency.
- **Channels** (`chan`) — synchronous or buffered message passing.
- **Global and local variables** — finite types only (`bit`, `byte`, `short`, `int`, `mtype`).
- **Guarded commands** — `if`/`fi` and `do`/`od`, with nondeterministic choice.
- **Blocking semantics** — a statement executes only when it is *executable*.

Important idea

In PROMELA a **boolean expression is a statement**. It blocks until it is true. So `(x > 0)` on its own line means "wait until `x > 0`".

PROMELA in one slide

Process Meta Language. C-like syntax, but it is a **modeling** language, not a programming language.

- **Processes** (`proctype`) — the units of concurrency.
- **Channels** (`chan`) — synchronous or buffered message passing.
- **Global and local variables** — finite types only (`bit`, `byte`, `short`, `int`, `mtype`).
- **Guarded commands** — `if`/`fi` and `do`/`od`, with nondeterministic choice.
- **Blocking semantics** — a statement executes only when it is *executable*.

Important idea

In PROMELA a **boolean expression is a statement**. It blocks until it is true. So `(x > 0)` on its own line means "wait until `x > 0`".

Processes

```
byte powered = 0;                /* global, shared */

proctype Instrument(byte id) {
  byte local = 0;                /* local to each instance */
  local = powered;
  powered = local + 1;
  printf("instrument %d on\n", id)
}

init {
  run Instrument(1);             /* spawn */
  run Instrument(2)
}
```

Statements of different processes interleave arbitrarily — **that is the point**.

Processes

```
byte powered = 0;                /* global, shared */

proctype Instrument(byte id) {
  byte local = 0;                /* local to each instance */
  local = powered;
  powered = local + 1;
  printf("instrument %d on\n", id)
}

init {
  run Instrument(1);             /* spawn */
  run Instrument(2)
}
```

Statements of different processes interleave arbitrarily — **that is the point**.

Guarded commands: `if`

```
/* the environment: temperature does what it likes */
if
:: (t < 120) → t = t + 10        /* it rises            */
:: (t >   0) → t = t - 10        /* it falls            */
:: true      → sensor = FAILED   /* or the sensor dies */
fi
```

Semantics

- Every guard is evaluated. **All** true guards are candidates.
- One of them is chosen **nondeterministically**. For $0 < t < 120$ all three hold, and SPIN explores **all three** branches.
- Guard order is irrelevant — this is not C's `else if`.
- If no guard is true and there is no `else`, the `if` **blocks**.

Nondeterminism is a modeling tool, not a bug. No branch here is the "right" one: whatever you prove holds for *every* environment behavior, including the sensor failing at the worst possible moment.

Guarded commands: `if`

```
/* the environment: temperature does what it likes */
if
:: (t < 120) → t = t + 10        /* it rises           */
:: (t >   0) → t = t - 10        /* it falls           */
:: true      → sensor = FAILED   /* or the sensor dies */
fi
```

Semantics

- Every guard is evaluated. **All** true guards are candidates.
- One of them is chosen **nondeterministically**. For $0 < t < 120$ all three hold, and SPIN explores **all three** branches.
- Guard order is irrelevant — this is not C's `else if`.
- If no guard is true and there is no `else`, the `if` **blocks**.

Nondeterminism is a modeling tool, not a bug. No branch here is the "right" one: whatever you prove holds for *every* environment behavior, including the sensor failing at the worst possible moment.

Guarded commands: `if`

```
/* the environment: temperature does what it likes */
if
:: (t < 120) → t = t + 10        /* it rises           */
:: (t >   0) → t = t - 10        /* it falls           */
:: true      → sensor = FAILED   /* or the sensor dies */
fi
```

Semantics

- Every guard is evaluated. **All** true guards are candidates.
- One of them is chosen **nondeterministically**. For $0 < t < 120$ all three hold, and SPIN explores **all three** branches.
- Guard order is irrelevant — this is not C's `else if`.
- If no guard is true and there is no `else`, the `if` **blocks**.

Nondeterminism is a modeling tool, not a bug. No branch here is the "right" one: whatever you prove holds for *every* environment behavior, including the sensor failing at the worst possible moment.

Guarded commands: do

```
/* the same environment, now running forever */
do
:: (t < 120) → t = t + 10
:: (t >   0) → t = t - 10
:: true      → sensor = FAILED; break
od
```

Same choice rule, repeated forever until `break` or `goto`. The temperature now wanders over every reachable value, and the sensor may fail at **any** point along the way.

Channels

```
mtype = { HEAT, COOL, ACK };            /* symbolic constants */

chan cmd  = [0] of { mtype, byte };     /* rendezvous  */
chan tlm  = [8] of { mtype, byte };     /* buffered, capacity 8 */

cmd ! HEAT, 42;       /* send    */

mtype msg; byte value;
tlm ? msg, value;     /* receive */
```

- Capacity `[0]` $\Rightarrow$ **synchronous** handshake: sender and receiver move together.
- Capacity > 0 $\Rightarrow$ **asynchronous** FIFO; send blocks when full, receive blocks when empty.
- `len(ch)`, `empty(ch)`, `full(ch)`, `nempty(ch)`, `nfull(ch)`.
- `ch ? eval(x)` matches only a message equal to the current value of `x`; `ch ? [msg, value]` tests receivability without consuming.

Channels

```
mtype = { HEAT, COOL, ACK };          /* symbolic constants */

chan cmd  = [0] of { mtype, byte };   /* rendezvous  */
chan tlm  = [8] of { mtype, byte };   /* buffered, capacity 8 */

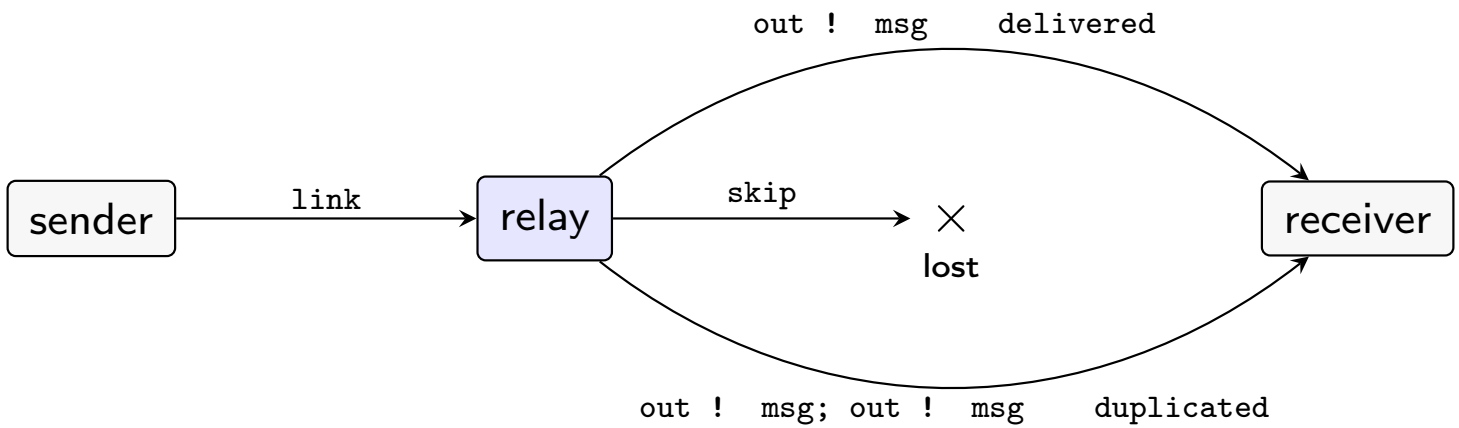
cmd ! HEAT, 42;       /* send    */

mtype msg; byte value;
tlm ? msg, value;     /* receive */
```

- Capacity `[0]` $\Rightarrow$ **synchronous** handshake: sender and receiver move together.
- Capacity > 0 $\Rightarrow$ **asynchronous** FIFO; send blocks when full, receive blocks when empty.
- `len(ch)`, `empty(ch)`, `full(ch)`, `nempty(ch)`, `nfull(ch)`.
- `ch ? eval(x)` matches only a message equal to the current value of `x`; `ch ? [msg, value]` tests receivability without consuming.

Modeling an unreliable channel

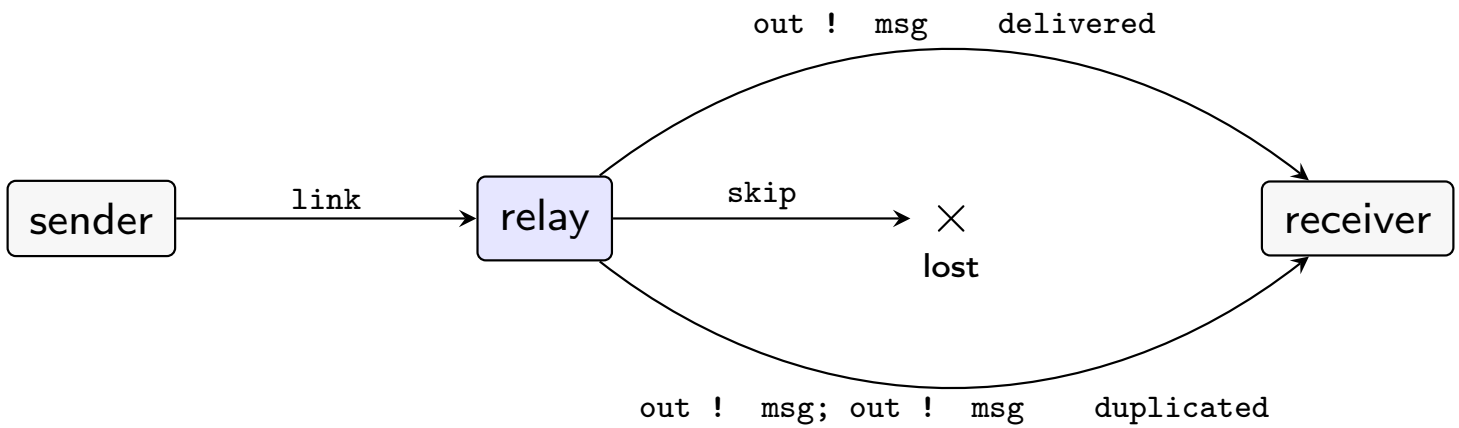
```
do
:: link ? msg →
     if
     :: out ! msg                  /* delivered    */
     :: skip                       /* silently lost */
     :: out ! msg; out ! msg       /* duplicated   */
     fi
od
```



Three lines of nondeterminism replace an entire fault-injection harness. Every property you then prove holds under loss and duplication.

Modeling an unreliable channel

```
do
:: link ? msg →
     if
     :: out ! msg                  /* delivered    */
     :: skip                       /* silently lost */
     :: out ! msg; out ! msg       /* duplicated   */
     fi
od
```



Three lines of nondeterminism replace an entire fault-injection harness. Every property you then prove holds under loss and duplication.

Atomicity

By default, **every statement is a possible preemption point**. Two ways to say otherwise:

```
atomic { t = x; x = t + 1 }   /* atomicity lost if it blocks */

d_step { t = x; x = t + 1 }   /* one step, may never block  */
```

Why you care

- `atomic` and `d_step` are how you model a critical section, a disabled-interrupt region, or a hardware-atomic instruction.
- They are also the main lever for **reducing the state space**: fewer interleaving points means exponentially fewer states.
- **Danger:** over-using them hides the very bugs you are looking for.

Atomicity

By default, **every statement is a possible preemption point**. Two ways to say otherwise:

```
atomic { t = x; x = t + 1 }   /* atomicity lost if it blocks */

d_step { t = x; x = t + 1 }   /* one step, may never block  */
```

Why you care

- `atomic` and `d_step` are how you model a critical section, a disabled-interrupt region, or a hardware-atomic instruction.
- They are also the main lever for **reducing the state space**: fewer interleaving points means exponentially fewer states.
- **Danger:** over-using them hides the very bugs you are looking for.

Stating properties

Two kinds of property

Safety — "nothing bad ever happens"

Violated by a **finite** execution prefix.
Both instruments powered simultaneously. An assertion fails. A deadlock.

Liveness — "something good eventually happens"

Violated only by an **infinite** execution.
A request is never granted. The system loops forever without progressing.

SPIN handles both, but by different machinery: safety by ordinary reachability, liveness by cycle detection. Liveness checking is roughly twice the cost.

Two kinds of property

Safety — "nothing bad ever happens"

Violated by a **finite** execution prefix.
Both instruments powered simultaneously. An assertion fails. A deadlock.

Liveness — "something good eventually happens"

Violated only by an **infinite** execution.
A request is never granted. The system loops forever without progressing.

SPIN handles both, but by different machinery: safety by ordinary reachability, liveness by cycle detection. Liveness checking is roughly twice the cost.

Two kinds of property

Safety — "nothing bad ever happens"

Violated by a **finite** execution prefix.
Both instruments powered simultaneously. An assertion fails. A deadlock.

Liveness — "something good eventually happens"

Violated only by an **infinite** execution.
A request is never granted. The system loops forever without progressing.

SPIN handles both, but by different machinery: safety by ordinary reachability, liveness by cycle detection. Liveness checking is roughly twice the cost.

Level 1: assertions

```
assert(powered <= 1);
assert(len(cmd_queue) < QUEUE_MAX);
assert(mode != SAFE_MODE || thrusters_off);
```

Global invariants

Put the invariant in its own monitor process; SPIN will interleave it everywhere:

```
active proctype Monitor() {
  assert(powered <= 1)      /* checked at all interleaving points */
}
```

Level 1: assertions

```
assert(powered <= 1);
assert(len(cmd_queue) < QUEUE_MAX);
assert(mode != SAFE_MODE || thrusters_off);
```

Global invariants

Put the invariant in its own monitor process; SPIN will interleave it everywhere:

```
active proctype Monitor() {
  assert(powered <= 1)      /* checked at all interleaving points */
}
```

Level 2: end states, progress, acceptance

Valid end states — `end` labels

By default a process must terminate to end validly. Label a legitimate blocking point to say "stopping here is fine":

```
end_wait: do :: cmd ? c → handle(c) od
```

Without the `end` prefix, an idle server looks like a deadlock.

Progress — `progress` labels

```
progress_grant: powered++;
```

With `./pan -l`, SPIN reports any infinite execution that does **not** pass a `progress` label infinitely often — i.e. livelock / starvation.

Level 2: end states, progress, acceptance

Valid end states — `end` labels

By default a process must terminate to end validly. Label a legitimate blocking point to say "stopping here is fine":

```
end_wait: do :: cmd ? c → handle(c) od
```

Without the `end` prefix, an idle server looks like a deadlock.

Progress — `progress` labels

```
progress_grant: powered++;
```

With `./pan -l`, SPIN reports any infinite execution that does **not** pass a `progress` label infinitely often — i.e. livelock / starvation.

Level 3: Linear Temporal Logic

Over an infinite execution $\sigma = s_0 s_1 s_2 \ldots$

Operator	SPIN **syntax**	**Meaning**
$\Box\, p$	`[] p`	p holds in every state
$\Diamond\, p$	`<> p`	p holds in some future state
$\bigcirc\, p$	`X p`	p holds in the next state
$p\, \mathcal{U}\, q$	`p U q`	p holds until q holds, and q does

The three patterns that cover most real requirements

Invariance $\Box\, \neg bad$

Response $\Box\, (request \rightarrow \Diamond\, grant)$

Precedence $\neg action\, \mathcal{U}\, authorization$

Level 3: Linear Temporal Logic

Over an infinite execution $\sigma = s_0 s_1 s_2 \ldots$

Operator	SPIN **syntax**	**Meaning**
$\Box\, p$	`[] p`	p holds in every state
$\Diamond\, p$	`<> p`	p holds in some future state
$\bigcirc\, p$	`X p`	p holds in the next state
$p\, \mathcal{U}\, q$	`p U q`	p holds until q holds, and q does

The three patterns that cover most real requirements

Invariance $\Box\, \neg bad$

Response $\Box\, (request \rightarrow \Diamond\, grant)$

Precedence $\neg action\, \mathcal{U}\, authorization$

LTL in Promela

```
#define p0  (want[0])           /* instrument 0 requests */
#define g0  (powered_by == 0)   /* instrument 0 has the bus */
#define mutex (powered <= 1)

ltl safety   { [] mutex }
ltl response { [] (p0 → <> g0) }
ltl no_starve { []<> g0 }
```

Worked examples

1. A recent analysis

A lock-free priority queue, August 2026

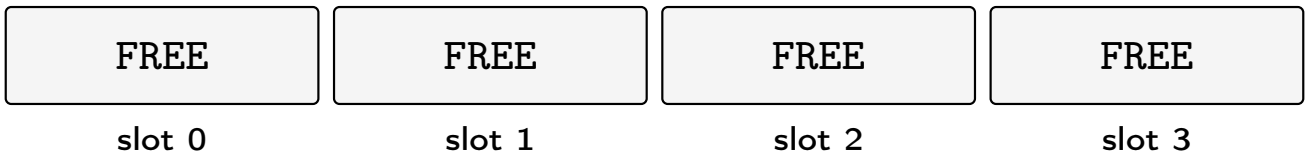
The queue

The problem

A message queue built on a mutex cannot be used from an interrupt handler, because a mutex cannot be taken there.

So: replace the mutex with compare-and-swap.

An array of `depth` **slots**. Each holds one message and an atomic word giving its state — all `FREE` to begin with:



`FREE` → `WRITING` → `READY` → `READING` → `FREE`

A producer takes a slot by winning the CAS `FREE`→`WRITING`; a consumer takes it by winning the CAS `READY`→`READING`. **Winning the CAS is what confers ownership** — two consumers may pick the same slot, only one wins, so nothing is delivered twice.

And one counter, purely for speed

Scanning the whole array every time, just to see whether there is an element, is costly — and it is asked every dispatch cycle.

The optimization: `available`

A counter, so a consumer can skip the scan when there are likely no elements.

The count now lives in **two places** — the counter, and implicitly the slots themselves. They need not match exactly: the counter is only a hint, and whoever passes it still runs the real scan. What matters is *which way* it is allowed to be wrong.

It is also public: `getMessagesAvailable()` is downlinked as telemetry.

And one counter, purely for speed

Scanning the whole array every time, just to see whether there is an element, is costly — and it is asked every dispatch cycle.

The optimization: `available`

A counter, so a consumer can skip the scan when there are likely no elements.

The count now lives in **two places** — the counter, and implicitly the slots themselves. They need not match exactly: the counter is only a hint, and whoever passes it still runs the real scan. What matters is *which way* it is allowed to be wrong.

It is also public: `getMessagesAvailable()` is downlinked as telemetry.

send and receive, as written

```
send(m):
    slot = claimFreeSlot()                  # CAS FREE -> WRITING
    write(slot, m)
    publish(slot)                           # CAS WRITING -> READY
    available += 1                          # credit it AFTER publishing

receive():
    repeat, at most MAX_RETRY_PASSES times:   # a pass = peek, scan, claim
        if available == 0:      continue      # 1. peek: skip the scan
        slot = selectBestReady()              # 2. scan: claims nothing
        if slot == none:        continue
        if CAS(slot, READY -> READING):       # 3. claim: fails if another
            available -= 1                    #    consumer got there first
            m = read(slot)
            release(slot)                     # CAS READING -> FREE
            return m
        # lost the claim -- yield and try again
    return EMPTY                              # all passes used up
```

No mutex means nothing to sleep on: a blocking caller **polls** the counter, not the array.

Only the third step commits to anything: the decrement is authorized by **winning that CAS**. Two consumers winning on *different* slots have both earned one.

send and receive, as written

```
send(m):
    slot = claimFreeSlot()                  # CAS FREE -> WRITING
    write(slot, m)
    publish(slot)                           # CAS WRITING -> READY
    available += 1                          # credit it AFTER publishing

receive():
    repeat, at most MAX_RETRY_PASSES times:   # a pass = peek, scan, claim
        if available == 0:      continue      # 1. peek: skip the scan
        slot = selectBestReady()              # 2. scan: claims nothing
        if slot == none:        continue
        if CAS(slot, READY -> READING):       # 3. claim: fails if another
            available -= 1                    #    consumer got there first
            m = read(slot)
            release(slot)                     # CAS READING -> FREE
            return m
        # lost the claim -- yield and try again
    return EMPTY                              # all passes used up
```

No mutex means nothing to sleep on: a blocking caller **polls** the counter, not the array.

Only the third step commits to anything: the decrement is authorized by **winning that CAS**. Two consumers winning on *different* slots have both earned one.

send and receive, as written

```
send(m):
    slot = claimFreeSlot()                  # CAS FREE -> WRITING
    write(slot, m)
    publish(slot)                           # CAS WRITING -> READY
    available += 1                          # credit it AFTER publishing

receive():
    repeat, at most MAX_RETRY_PASSES times:   # a pass = peek, scan, claim
        if available == 0:      continue      # 1. peek: skip the scan
        slot = selectBestReady()              # 2. scan: claims nothing
        if slot == none:        continue
        if CAS(slot, READY -> READING):       # 3. claim: fails if another
            available -= 1                    #    consumer got there first
            m = read(slot)
            release(slot)                     # CAS READING -> FREE
            return m
        # lost the claim -- yield and try again
    return EMPTY                              # all passes used up
```

No mutex means nothing to sleep on: a blocking caller **polls** the counter, not the array.

Only the third step commits to anything: the decrement is authorized by **winning that CAS**. Two consumers winning on *different* slots have both earned one.

The model — the producer

Claude Code flagged an issue. I then asked it to generate a PROMELA model to illustrate it.

```
/* FREE=0, WRITING=1, READY=2, READING=3 */
byte slot[2] = FREE;                    /* each slot's state */
int  avail = 0;                         /* signed: underflow shows as -1 */
active [2] proctype Producer() {
  int myslot;                           /* the slot this producer claimed */
  do
  :: atomic {                           /* the claim CAS */
       if
       :: (slot[0] == FREE) → slot[0] = WRITING; myslot = 0
       :: (slot[1] == FREE) → slot[1] = WRITING; myslot = 1
       fi
     }
     if
     :: (myslot == 0) → slot[0] = READY
     :: (myslot == 1) → slot[1] = READY
     fi;
     avail++                            /* publish, THEN credit */
  od
}
```

The model — the consumer

```
active [2] proctype Consumer() {
  int myslot;                          /* -1 if nothing was claimed */
  do
  :: avail > 0 →                       /* the peek: a step of its own */
     atomic {                          /* the claim CAS */
       if
       :: (slot[0] == READY) → slot[0] = READING; myslot = 0; avail--
       :: (slot[1] == READY) → slot[1] = READING; myslot = 1; avail--
       :: else               → myslot = -1
       fi
     }
     assert(avail >= 0);
     if
     :: (myslot ==  0) → slot[0] = FREE    /* release: READING → FREE */
     :: (myslot ==  1) → slot[1] = FREE
     :: (myslot == -1) → skip              /* claimed nothing */
     fi
  od
}
```

Two producers, two consumers, two slots; of a slot, only its **state** is modelled.

The crux: publish and `avail++` are separate statements, and so are the peek and the claim. Put either pair inside one `atomic` and the bug vanishes.

The model — the consumer

```
active [2] proctype Consumer() {
  int myslot;                          /* -1 if nothing was claimed */
  do
  :: avail > 0 →                       /* the peek: a step of its own */
     atomic {                          /* the claim CAS */
       if
       :: (slot[0] == READY) → slot[0] = READING; myslot = 0; avail--
       :: (slot[1] == READY) → slot[1] = READING; myslot = 1; avail--
       :: else               → myslot = -1
       fi
     }
     assert(avail >= 0);
     if
     :: (myslot ==  0) → slot[0] = FREE    /* release: READING → FREE */
     :: (myslot ==  1) → slot[1] = FREE
     :: (myslot == -1) → skip              /* claimed nothing */
     fi
  od
}
```

Two producers, two consumers, two slots; of a slot, only its **state** is modelled. The crux: publish and `avail++` are separate statements, and so are the peek and the claim. Put either pair inside one `atomic` and the bug vanishes.

What SPIN says

```
pan:1: assertion violated (avail>=0) (at depth 11)
```

step	proc	action	avail
1	P	claim slot 0	0
3	P	publish slot 0 → `READY`	0
4	P	`avail++`	1
5	C1	peek: `avail > 0`, pass	1
6	C2	peek: `avail > 0`, pass	1
7	C1	claim slot 0, `avail--`	0
8	P	claim slot 1	0
10	P	publish slot 1 — *uncredited*	0
11	C2	claim slot 1, `avail--`	−1
12	`assert(avail >= 0)` fails		

Both consumers won a claim, on *different* slots, so both were entitled to decrement — but only one credit had landed. The real field is `unsigned`: telemetry reads **4 294 967 295**.

Mirror symptom, same gap: the peek reads 0 past a published message, so a non-blocking `receive` runs out its passes and returns `EMPTY` with the queue non-empty — a **linearizability violation**.

What SPIN says

```
pan:1: assertion violated (avail>=0) (at depth 11)
```

step	proc	action	avail
1	P	claim slot 0	0
3	P	publish slot 0 → `READY`	0
4	P	`avail++`	1
5	C1	peek: `avail > 0`, pass	1
6	C2	peek: `avail > 0`, pass	1
7	C1	claim slot 0, `avail--`	0
8	P	claim slot 1	0
10	P	publish slot 1 — *uncredited*	0
11	C2	claim slot 1, `avail--`	−1
12	`assert(avail >= 0)` fails		

Both consumers won a claim, on *different* slots, so both were entitled to decrement — but only one credit had landed. The real field is `unsigned`: telemetry reads **4 294 967 295**.

Mirror symptom, same gap: the peek reads 0 past a published message, so a non-blocking `receive` runs out its passes and returns `EMPTY` with the queue non-empty — a **linearizability violation**.

What SPIN says

```
pan:1: assertion violated (avail>=0) (at depth 11)
```

step	proc	action	avail
1	P	claim slot 0	0
3	P	publish slot 0 → `READY`	0
4	P	`avail++`	1
5	C1	peek: `avail > 0`, pass	1
6	C2	peek: `avail > 0`, pass	1
7	C1	claim slot 0, `avail--`	0
8	P	claim slot 1	0
10	P	publish slot 1 — *uncredited*	0
11	C2	claim slot 1, `avail--`	−1
12	`assert(avail >= 0)` fails		

Both consumers won a claim, on *different* slots, so both were entitled to decrement — but only one credit had landed. The real field is `unsigned`: telemetry reads **4 294 967 295**.

Mirror symptom, same gap: the peek reads 0 past a published message, so a non-blocking `receive` runs out its passes and returns `EMPTY` with the queue non-empty — a **linearizability violation**.

The fix

```
send(m):
    slot = claimFreeSlot()
    write(slot, m)
    available += 1                  # credit it BEFORE publishing
    publish(slot)                   # CAS WRITING -> READY
```

Every decrement is now preceded by its own credit, so underflow is unreachable — and `available == 0` soundly implies no slot is `READY`, which kills the spurious `EMPTY` too.

```
State-vector 48 byte, depth reached 1097, errors: 0
     1992 states, stored
```

The trade-off, stated honestly

With two racing atomics you cannot have both *never over-report* and *never underflow*. Crediting first over-reports by at most `depth` — benign, because the peek was only ever a hint: a consumer that passes it still runs the real scan, finds nothing, and gives up.

The fix

```
send(m):
    slot = claimFreeSlot()
    write(slot, m)
    available += 1                  # credit it BEFORE publishing
    publish(slot)                   # CAS WRITING -> READY
```

Every decrement is now preceded by its own credit, so underflow is unreachable — and `available == 0` soundly implies no slot is `READY`, which kills the spurious `EMPTY` too.

```
State-vector 48 byte, depth reached 1097, errors: 0
     1992 states, stored
```

The trade-off, stated honestly

With two racing atomics you cannot have both *never over-report* and *never underflow*. Crediting first over-reports by at most `depth` — benign, because the peek was only ever a hint: a consumer that passes it still runs the real scan, finds nothing, and gives up.

Why the tests did not catch it

- The author's tests pass: **29/29** functional, **15/15** under ThreadSanitizer, **zero** race reports.
- It *is* a race — but a **high-level** one, across two variables: the publish and the credit. Not a data race on one location. Every individual access is a properly synchronized atomic, so ThreadSanitizer has nothing to see.
- The stress tests assert exactly-once delivery, which genuinely holds. Nothing asserted `available` $\leq$ `depth`.

The part worth pausing on

`available` did not exist in the original PR. It was *added* to fix an earlier reviewer's finding — and the ordering that reviewer recommended, faithfully implemented, is the ordering that underflows.

The fix for the first problem created this one.

Why the tests did not catch it

- The author's tests pass: **29/29** functional, **15/15** under ThreadSanitizer, **zero** race reports.
- It *is* a race — but a **high-level** one, across two variables: the publish and the credit. Not a data race on one location. Every individual access is a properly synchronized atomic, so ThreadSanitizer has nothing to see.
- The stress tests assert exactly-once delivery, which genuinely holds. Nothing asserted `available` $\leq$ `depth`.

The part worth pausing on

`available` did not exist in the original PR. It was *added* to fix an earlier reviewer's finding — and the ordering that reviewer recommended, faithfully implemented, is the ordering that underflows.

The fix for the first problem created this one.

Worked examples

2. The Deep Space 1 Remote Agent

Havelund, Lowry & Penix, NASA Ames — *Formal Analysis of a Space Craft Controller using* SPIN

The system

The **Remote Agent** is an AI-based spacecraft controller: a planner, a plan **execution module**, and a mode identification and recovery module. It flew on Deep Space 1, launched October 1998, and in May 1999 controlled a spacecraft autonomously for the first time in NASA's history.

We analyzed the executive's bottom layer — **ESL**, a language built on multithreaded Common Lisp. Tasks *achieve* and *maintain* properties of the spacecraft, take **locks** on them to exclude incompatible tasks, and a **daemon** watches for a locked property no longer holding.

The model

About **3000 lines** of Lisp, hand-translated to **500 lines** of PROMELA — abstracted down to the lock-table operations, with two property names, two events, and three tasks. Twelve man-weeks to build; about a week to verify.

The system

The **Remote Agent** is an AI-based spacecraft controller: a planner, a plan **execution module**, and a mode identification and recovery module. It flew on Deep Space 1, launched October 1998, and in May 1999 controlled a spacecraft autonomously for the first time in NASA's history.

We analyzed the executive's bottom layer — **ESL**, a language built on multithreaded Common Lisp. Tasks *achieve* and *maintain* properties of the spacecraft, take **locks** on them to exclude incompatible tasks, and a **daemon** watches for a locked property no longer holding.

The model

About **3000 lines** of Lisp, hand-translated to **500 lines** of PROMELA — abstracted down to the lock-table operations, with two property names, two events, and three tasks. Twelve man-weeks to build; about a week to verify.

The system

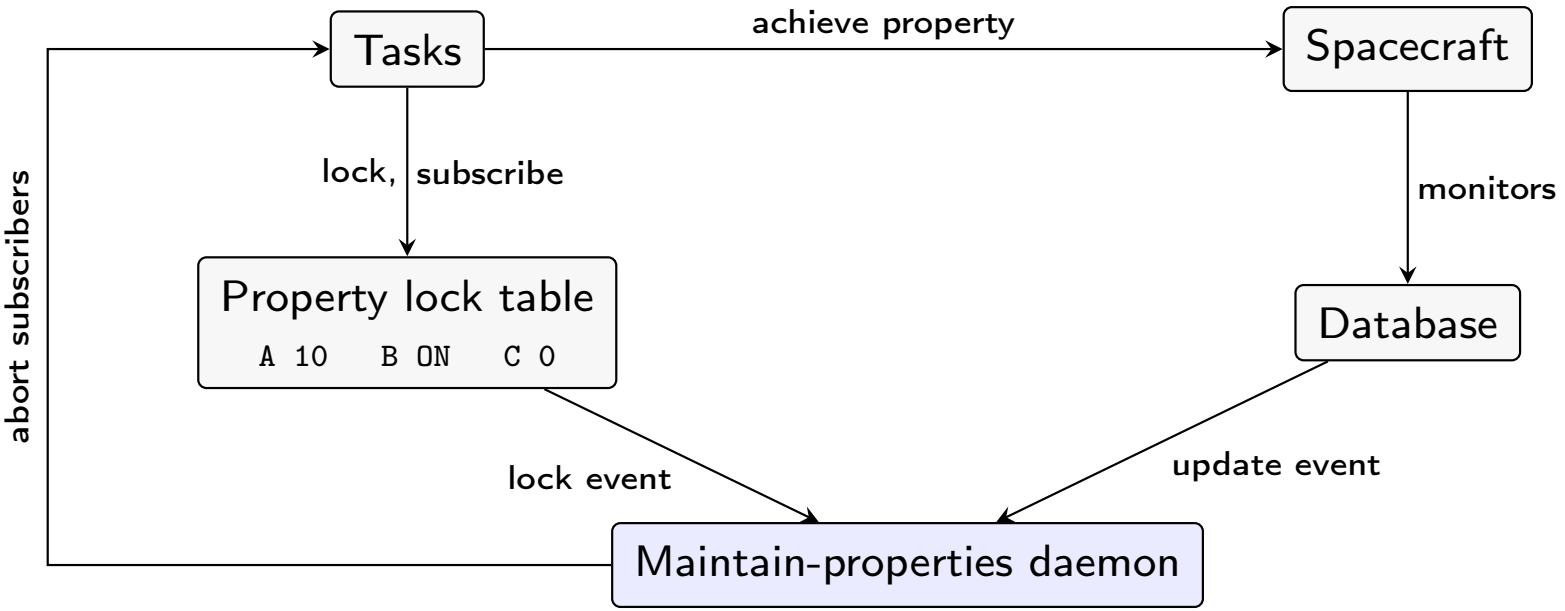
The **Remote Agent** is an AI-based spacecraft controller: a planner, a plan **execution module**, and a mode identification and recovery module. It flew on Deep Space 1, launched October 1998, and in May 1999 controlled a spacecraft autonomously for the first time in NASA's history.

We analyzed the executive's bottom layer — **ESL**, a language built on multithreaded Common Lisp. Tasks *achieve* and *maintain* properties of the spacecraft, take **locks** on them to exclude incompatible tasks, and a **daemon** watches for a locked property no longer holding.

The model

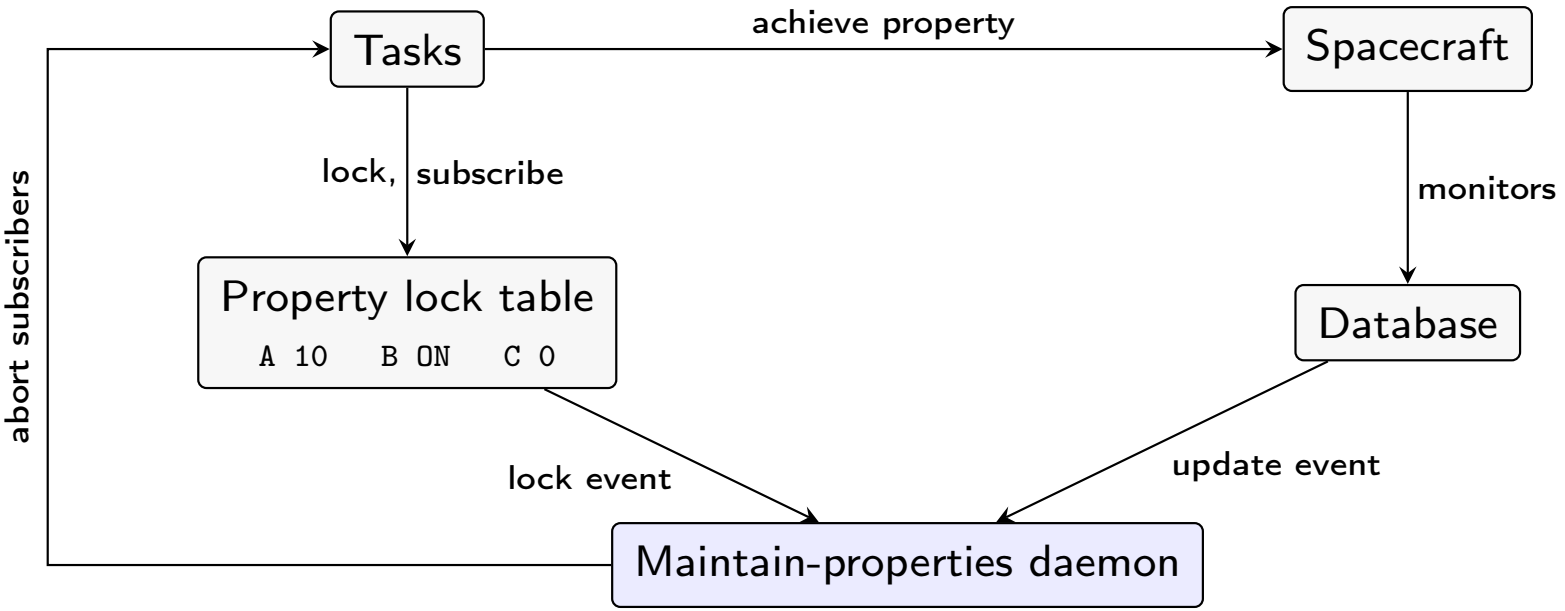
About **3000 lines** of Lisp, hand-translated to **500 lines** of PROMELA — abstracted down to the lock-table operations, with two property names, two events, and three tasks. Twelve man-weeks to build; about a week to verify.

The Remote Agent Executive



A task locks a property, then achieves it — writing it into the spacecraft. The **database** is what the monitors actually observe. When the lock table says a property is held and achieved but the database disagrees, that is an **inconsistency**, and the daemon must abort every task subscribed to it.

The Remote Agent Executive



A task locks a property, then achieves it — writing it into the spacecraft. The **database** is what the monitors actually observe. When the lock table says a property is held and achieved but the database disagrees, that is an **inconsistency**, and the daemon must abort every task subscribed to it.

Two properties, written by the programmers

Crucially, the RA programmers formulated these *after* the PROMELA model had been written — so the properties had no influence on the design of the model, and the model was not built to satisfy them.

RELEASE

A task releases all of its locks before it terminates.
An assertion, placed after the task's body.

ABORT

If an inconsistency occurs between the database and an entry in the lock table, then all tasks that rely on the lock will be terminated.

```
[] (task1_property_broken → <> task1_terminated)
```

A **response** property — the second of the three patterns from Part 3.

Both turned out to be broken. Five flawed code fragments: one breaking RELEASE, three breaking ABORT, one efficiency bug. The programmer confirmed all five were errors in the *Lisp*, not just in the model.

Two properties, written by the programmers

Crucially, the RA programmers formulated these *after* the PROMELA model had been written — so the properties had no influence on the design of the model, and the model was not built to satisfy them.

RELEASE

A task releases all of its locks before it terminates.
An assertion, placed after the task's body.

ABORT

If an inconsistency occurs between the database and an entry in the lock table, then all tasks that rely on the lock will be terminated.

```
[] (task1_property_broken → <> task1_terminated)
```

A **response** property — the second of the three patterns from Part 3.

Both turned out to be broken. Five flawed code fragments: one breaking RELEASE, three breaking ABORT, one efficiency bug. The programmer confirmed all five were errors in the *Lisp*, not just in the model.

Two properties, written by the programmers

Crucially, the RA programmers formulated these *after* the PROMELA model had been written — so the properties had no influence on the design of the model, and the model was not built to satisfy them.

RELEASE

A task releases all of its locks before it terminates.
An assertion, placed after the task's body.

ABORT

If an inconsistency occurs between the database and an entry in the lock table, then all tasks that rely on the lock will be terminated.

```
[] (task1_property_broken → <> task1_terminated)
```

A **response** property — the second of the three patterns from Part 3.

Both turned out to be broken. Five flawed code fragments: one breaking RELEASE, three breaking ABORT, one efficiency bug. The programmer confirmed all five were errors in the *Lisp*, not just in the model.

Error 2 — the one that mattered

Three counters:

`Ev[MEMORY_EVENT].count` — how many times the database has changed.
`Ev[LOCK_EVENT].count` — how many times the lock table has changed.
`event_count` — the daemon's snapshot of their sum, last time it looked.

```
if                                  /* has anything happened since I looked? */
:: (!first_time &&
    Ev[MEMORY_EVENT].count +
    Ev[LOCK_EVENT].count != event_count) →
      event_count = Ev[MEMORY_EVENT].count +
                    Ev[LOCK_EVENT].count
:: else →                           /* no: go to sleep */
      first_time = false;
      /* <-- an event arriving HERE is lost */
      wait_for_events(this, MEMORY_EVENT, LOCK_EVENT)
fi
```

The guard reads the event counters, the `else` branch commits to sleeping — and the sleeping happens later, as a **separate statement**.

Error 2 — the one that mattered

Three counters:

`Ev[MEMORY_EVENT].count` — how many times the database has changed.
`Ev[LOCK_EVENT].count` — how many times the lock table has changed.
`event_count` — the daemon's snapshot of their sum, last time it looked.

```
if                                  /* has anything happened since I looked? */
:: (!first_time &&
    Ev[MEMORY_EVENT].count +
    Ev[LOCK_EVENT].count != event_count) →
      event_count = Ev[MEMORY_EVENT].count +
                    Ev[LOCK_EVENT].count
:: else →                           /* no: go to sleep */
      first_time = false;
      /* <-- an event arriving HERE is lost */
      wait_for_events(this, MEMORY_EVENT, LOCK_EVENT)
fi
```

The guard reads the event counters, the `else` branch commits to sleeping — and the sleeping happens later, as a **separate statement**.

Error 2 — the fix

```
atomic {                              /* deciding and sleeping, indivisibly */
  if
  :: (!first_time && ...) → event_count = ...
  :: else → first_time = false;
             wait_for_events(this, MEMORY_EVENT, LOCK_EVENT)
  fi
}
```

One keyword. The daemon can no longer be preempted between deciding that nothing has happened and going to sleep.

The same shape as the queue. There it was publish-then-credit; here it is check-then-wait. Both are a test and the act that depends on it, performed as two steps — and in both cases the whole bug is the gap between them.

Error 2 — the fix

```
atomic {                              /* deciding and sleeping, indivisibly */
  if
  :: (!first_time && ...) → event_count = ...
  :: else → first_time = false;
             wait_for_events(this, MEMORY_EVENT, LOCK_EVENT)
  fi
}
```

One keyword. The daemon can no longer be preempted between deciding that nothing has happened and going to sleep.

The same shape as the queue. There it was publish-then-credit; here it is check-then-wait. Both are a test and the act that depends on it, performed as two steps — and in both cases the whole bug is the gap between them.

18 May 1999

Deep Space 1, 96 million kilometres from Earth. **Thrusting did not turn off when requested.** The Remote Agent experiment was terminated from the ground and the spacecraft put into stand-by mode.

The cause was a deadlock: a missing critical section around a conditional wait, letting an event arrive between the evaluation of the condition and the wait. The thread waited, missed the event, and the system wedged.

The same defect as Error 2 — and not by coincidence. The code had been **copied** into another module, and the critical section around it was, by mistake, left out of the copy. That module was never analyzed with SPIN.

The repair was to put the critical section back — the fix SPIN's error trace had already produced, months earlier, for the same pattern elsewhere.

18 May 1999

Deep Space 1, 96 million kilometres from Earth. **Thrusting did not turn off when requested.** The Remote Agent experiment was terminated from the ground and the spacecraft put into stand-by mode.

The cause was a deadlock: a missing critical section around a conditional wait, letting an event arrive between the evaluation of the condition and the wait. The thread waited, missed the event, and the system wedged.

The same defect as Error 2 — and not by coincidence. The code had been **copied** into another module, and the critical section around it was, by mistake, left out of the copy. That module was never analyzed with SPIN.

The repair was to put the critical section back — the fix SPIN's error trace had already produced, months earlier, for the same pattern elsewhere.

18 May 1999

Deep Space 1, 96 million kilometres from Earth. **Thrusting did not turn off when requested.** The Remote Agent experiment was terminated from the ground and the spacecraft put into stand-by mode.

The cause was a deadlock: a missing critical section around a conditional wait, letting an event arrive between the evaluation of the condition and the wait. The thread waited, missed the event, and the system wedged.

The same defect as Error 2 — and not by coincidence. The code had been **copied** into another module, and the critical section around it was, by mistake, left out of the copy. That module was never analyzed with SPIN.

The repair was to put the critical section back — the fix SPIN's error trace had already produced, months earlier, for the same pattern elsewhere.

What the programmer said

On Error 1:

"This is a particularly pernicious bug. It arises only because you are in a multi-threaded environment, and only in very obscure circumstances that are very unlikely to arise during testing. Congratulations! You have just converted me into a believer in formal methods."

On Error 2 — the pattern that later flew:

"This is an instance of a classic pattern: not wrapping a conditional wait-for-events inside a critical section. This sort of mistake is very easy to make and happens all the time in our code. Thanks for catching this one!"

Afterwards:

"I used to be very skeptical of the utility of formal methods. . . what you have been doing is finding places where software violates design assumptions, which is not the same thing as proving correctness. To me you have demonstrated the utility of this approach beyond any question."

What the programmer said

On Error 1:

"This is a particularly pernicious bug. It arises only because you are in a multi-threaded environment, and only in very obscure circumstances that are very unlikely to arise during testing. Congratulations! You have just converted me into a believer in formal methods."

On Error 2 — the pattern that later flew:

"This is an instance of a classic pattern: not wrapping a conditional wait-for-events inside a critical section. This sort of mistake is very easy to make and happens all the time in our code. Thanks for catching this one!"

Afterwards:

"I used to be very skeptical of the utility of formal methods. . . what you have been doing is finding places where software violates design assumptions, which is not the same thing as proving correctness. To me you have demonstrated the utility of this approach beyond any question."

What the programmer said

On Error 1:

> *"This is a particularly pernicious bug. It arises only because you are in a multi-threaded environment, and only in very obscure circumstances that are very unlikely to arise during testing. Congratulations! You have just converted me into a believer in formal methods."*

On Error 2 — the pattern that later flew:

> *"This is an instance of a classic pattern: not wrapping a conditional wait-for-events inside a critical section. This sort of mistake is very easy to make and happens all the time in our code. Thanks for catching this one!"*

Afterwards:

> *"I used to be very skeptical of the utility of formal methods... what you have been doing is finding places where software violates design assumptions, which is not the same thing as proving correctness. To me you have demonstrated the utility of this approach beyond any question."*

How SPIN works

The automata-theoretic approach

1. The PROMELA model is an automaton A_M whose language $\mathcal{L}(A_M)$ is the set of all its executions.
2. The negated property $\neg\varphi$ is translated into a Büchi automaton $A_{\neg\varphi}$ accepting exactly the **violating** executions.
3. SPIN builds the **synchronous product** $A_M \otimes A_{\neg\varphi}$.
4. Then:

$$M \models \varphi \iff \mathcal{L}(A_M \otimes A_{\neg\varphi}) = \emptyset$$

Verification = **emptiness checking**. And a Büchi automaton is non-empty iff some accepting state is reachable *and* lies on a cycle.

So: **safety** = find a bad state. **Liveness** = find a bad cycle.

The automata-theoretic approach

1. The PROMELA model is an automaton A_M whose language $\mathcal{L}(A_M)$ is the set of all its executions.
2. The negated property $\neg\varphi$ is translated into a Büchi automaton $A_{\neg\varphi}$ accepting exactly the **violating** executions.
3. SPIN builds the **synchronous product** $A_M \otimes A_{\neg\varphi}$.
4. Then:

$$M \models \varphi \iff \mathcal{L}(A_M \otimes A_{\neg\varphi}) = \emptyset$$

Verification = **emptiness checking**. And a Büchi automaton is non-empty iff some accepting state is reachable *and* lies on a cycle.

So: **safety** = find a bad state. **Liveness** = find a bad cycle.

The automata-theoretic approach

1. The PROMELA model is an automaton A_M whose language $\mathcal{L}(A_M)$ is the set of all its executions.
2. The negated property $\neg\varphi$ is translated into a Büchi automaton $A_{\neg\varphi}$ accepting exactly the **violating** executions.
3. SPIN builds the **synchronous product** $A_M \otimes A_{\neg\varphi}$.
4. Then:

$$M \models \varphi \iff \mathcal{L}(A_M \otimes A_{\neg\varphi}) = \emptyset$$

Verification = **emptiness checking**. And a Büchi automaton is non-empty iff some accepting state is reachable *and* lies on a cycle.

So: **safety** = find a bad state. **Liveness** = find a bad cycle.

The automata-theoretic approach

1. The PROMELA model is an automaton A_M whose language $\mathcal{L}(A_M)$ is the set of all its executions.
2. The negated property $\neg\varphi$ is translated into a Büchi automaton $A_{\neg\varphi}$ accepting exactly the **violating** executions.
3. SPIN builds the **synchronous product** $A_M \otimes A_{\neg\varphi}$.
4. Then:

$$M \models \varphi \quad \Longleftrightarrow \quad \mathcal{L}(A_M \otimes A_{\neg\varphi}) = \emptyset$$

Verification = **emptiness checking**. And a Büchi automaton is non-empty iff some accepting state is reachable *and* lies on a cycle.

So: **safety** = find a bad state. **Liveness** = find a bad cycle.

The automata-theoretic approach

1. The PROMELA model is an automaton A_M whose language $\mathcal{L}(A_M)$ is the set of all its executions.
2. The negated property $\neg\varphi$ is translated into a Büchi automaton $A_{\neg\varphi}$ accepting exactly the **violating** executions.
3. SPIN builds the **synchronous product** $A_M \otimes A_{\neg\varphi}$.
4. Then:

$$M \models \varphi \quad \Longleftrightarrow \quad \mathcal{L}(A_M \otimes A_{\neg\varphi}) = \emptyset$$

Verification = **emptiness checking**. And a Büchi automaton is non-empty iff some accepting state is reachable *and* lies on a cycle.

So: **safety** = find a bad state. **Liveness** = find a bad cycle.

The search

Safety: depth-first reachability

Push the initial state; expand successors; hash each into a table; report an error the first time an assertion fails, an end state is invalid, or a deadlock occurs. The DFS stack **is** the counterexample.

Liveness: a second search, hunting for a loop

A liveness violation is an *infinite* bad run — and in a finite model, an infinite run means a **loop**. The property automaton marks a state *accepting* when the violation is under way and could go on forever — for `[]<> g0` that is "g0 has not held since". So search as usual, and each time you finish exploring an accepting state, start a **second** search from it asking one question: *can I get back here?* If it can, that loop *is* the counterexample.

The *nested* depth-first search: linear time, two bits per state — seen by the first, seen by the second.

On-the-fly: SPIN never builds the state graph first — it constructs states as it searches and stops at the first violation. A bug in a 10^{20}-state system can surface after 10^3 states.

The search

Safety: depth-first reachability

Push the initial state; expand successors; hash each into a table; report an error the first time an assertion fails, an end state is invalid, or a deadlock occurs. The DFS stack **is** the counterexample.

Liveness: a second search, hunting for a loop

A liveness violation is an *infinite* bad run — and in a finite model, an infinite run means a **loop**. The property automaton marks a state *accepting* when the violation is under way and could go on forever — for `[]<> g0` that is "`g0` has not held since". So search as usual, and each time you finish exploring an accepting state, start a **second** search from it asking one question: *can I get back here?* If it can, that loop *is* the counterexample.

The *nested* depth-first search: linear time, two bits per state — seen by the first, seen by the second.

On-the-fly: SPIN never builds the state graph first — it constructs states as it searches and stops at the first violation. A bug in a 10^{20}-state system can surface after 10^3 states.

The search

Safety: depth-first reachability

Push the initial state; expand successors; hash each into a table; report an error the first time an assertion fails, an end state is invalid, or a deadlock occurs. The DFS stack **is** the counterexample.

Liveness: a second search, hunting for a loop

A liveness violation is an *infinite* bad run — and in a finite model, an infinite run means a **loop**. The property automaton marks a state *accepting* when the violation is under way and could go on forever — for `[]<> g0` that is "`g0` has not held since". So search as usual, and each time you finish exploring an accepting state, start a **second** search from it asking one question: *can I get back here?* If it can, that loop *is* the counterexample.

The *nested* depth-first search: linear time, two bits per state — seen by the first, seen by the second.

On-the-fly: SPIN never builds the state graph first — it constructs states as it searches and stops at the first violation. A bug in a 10^{20}-state system can surface after 10^3 states.

The enemy: state explosion

The number of states is roughly

$$|S| \approx \prod_i |P_i| \times \prod_j |V_j| \times \prod_k |C_k|$$

— process locations $\times$ variable valuations $\times$ channel contents.

Everything is multiplicative. One extra `byte` variable multiplies the state space by up to 256. One extra process multiplies it by that process's entire local state space.

Practical implication

Model-checking effort is dominated not by writing PROMELA but by **choosing what to leave out**.

The enemy: state explosion

The number of states is roughly

$$|S| \approx \prod_i |P_i| \times \prod_j |V_j| \times \prod_k |C_k|$$

— process locations $\times$ variable valuations $\times$ channel contents.

Everything is multiplicative. One extra `byte` variable multiplies the state space by up to 256. One extra process multiplies it by that process's entire local state space.

Practical implication

Model-checking effort is dominated not by writing PROMELA but by **choosing what to leave out**.

The enemy: state explosion

The number of states is roughly

$$|S| \approx \prod_i |P_i| \times \prod_j |V_j| \times \prod_k |C_k|$$

— process locations $\times$ variable valuations $\times$ channel contents.

Everything is multiplicative. One extra `byte` variable multiplies the state space by up to 256. One extra process multiplies it by that process's entire local state space.

Practical implication

Model-checking effort is dominated not by writing PROMELA but by **choosing what to leave out**.

SPIN's weapons against state explosion

- **Skip the orders that cannot matter** (*partial-order reduction*, on by default). If two processes are doing things that cannot affect each other, try one order and not the other. Usually an order of magnitude, free.
- **Pack each state tighter** (`-DCOLLAPSE`). Slower, smaller, exact.
- **Do not keep the states — keep hashes of them.** One idea at two settings; either can miss a bug, neither can invent one.
  - a **fingerprint** per state (`-DHC`): a few bytes each, so memory still grows with the number of states. Errs only if two states collide.
  - **no entries at all** (`-DBITSTATE`): one fixed-size scoreboard — tick a few boxes per state, "already ticked" means "been here". A few bits rather than 384, whatever the state's size. Errs once other states have ticked those boxes between them.
- **Search in parallel** (`-DNCORE=N`, *swarm*): many short searches, each looking elsewhere. Good at finding bugs; not a proof.
- **Give up past a chosen depth** (`-mN`). If the run reports `depth reached` equal to your bound it never finished, and `errors: 0` means nothing.

Rule of thumb: exhaustive if it fits; otherwise a lossy search that finds the bug beats an exhaustive one that runs out of memory.

SPIN's weapons against state explosion

- **Skip the orders that cannot matter** (*partial-order reduction*, on by default). If two processes are doing things that cannot affect each other, try one order and not the other. Usually an order of magnitude, free.
- **Pack each state tighter** (`-DCOLLAPSE`). Slower, smaller, exact.
- **Do not keep the states — keep hashes of them.** One idea at two settings; either can miss a bug, neither can invent one.
  - a **fingerprint** per state (`-DHC`): a few bytes each, so memory still grows with the number of states. Errs only if two states collide.
  - **no entries at all** (`-DBITSTATE`): one fixed-size scoreboard — tick a few boxes per state, "already ticked" means "been here". A few bits rather than 384, whatever the state's size. Errs once other states have ticked those boxes between them.
- **Search in parallel** (`-DNCORE=N`, *swarm*): many short searches, each looking elsewhere. Good at finding bugs; not a proof.
- **Give up past a chosen depth** (`-mN`). If the run reports `depth reached` equal to your bound it never finished, and `errors: 0` means nothing.

Rule of thumb: exhaustive if it fits; otherwise a lossy search that finds the bug beats an exhaustive one that runs out of memory.

Using the tool

The workflow

```
# 1. Sanity: random simulation, watch it run
spin model.pml

# 2. Generate the verifier (C source, specialized to this model)
spin -a model.pml                    # produces pan.c, pan.h, ...

# 3. Compile it
gcc -O2 -o pan pan.c

# 4. Verify
./pan                                # safety: assertions, deadlock
./pan -a                             # + liveness / acceptance cycles
./pan -a -f                          # + weak fairness
./pan -l                             # non-progress cycles

# 5. Replay the counterexample
spin -t -p -l -g model.pml
```

The key insight: SPIN is a *verifier generator*. It emits a C program tailored to your model, which is why it is fast.

The workflow

```
# 1. Sanity: random simulation, watch it run
spin model.pml

# 2. Generate the verifier (C source, specialized to this model)
spin -a model.pml                    # produces pan.c, pan.h, ...

# 3. Compile it
gcc -O2 -o pan pan.c

# 4. Verify
./pan                                # safety: assertions, deadlock
./pan -a                             # + liveness / acceptance cycles
./pan -a -f                          # + weak fairness
./pan -l                             # non-progress cycles

# 5. Replay the counterexample
spin -t -p -l -g model.pml
```

The key insight: SPIN is a *verifier generator*. It emits a C program tailored to your model, which is why it is fast.

Reading the output

```
(Spin Version 6.5.2)
        + Partial Order Reduction

Full statespace search for:
        never claim             - (none specified)
        assertion violations    +
        acceptance   cycles     - (not selected)
        invalid end states      +

State-vector 44 byte, depth reached 178, errors: 0
      1897 states, stored
       912 states, matched
      2809 transitions (= stored+matched)
         0 atomic steps
hash conflicts:         0 (resolved)

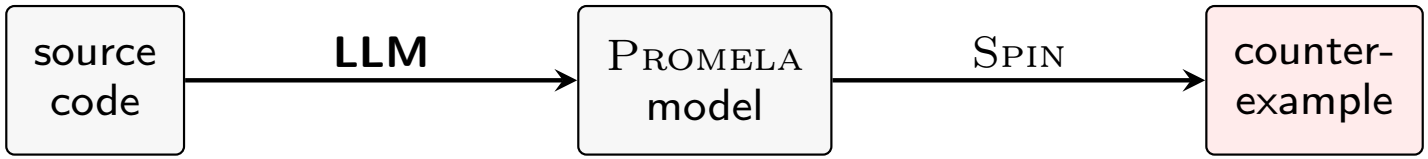
Stats on memory usage (in Megabytes):
    0.130       equivalent memory usage for states
  128.195       total actual memory usage
```

Reading a counterexample trail

```
$ spin -t -p lockless.pml                                    (abridged)

  4: proc 1 (Producer:1) lockless.pml:68 (state 16) [avail = (avail+1)]
  5: proc 3 (Consumer:1) lockless.pml:76 (state 1)  [((avail>0))]
  6: proc 2 (Consumer:1) lockless.pml:76 (state 1)  [((avail>0))]
  7: proc 3 (Consumer:1) lockless.pml:79 (state 3)  [slot[0] = 3]
  7: proc 3 (Consumer:1) lockless.pml:79 (state 5)  [avail = (avail-1)]
 10: proc 1 (Producer:1) lockless.pml:66 (state 13) [slot[1] = 2]
 11: proc 2 (Consumer:1) lockless.pml:80 (state 9)  [avail = (avail-1)]
spin: lockless.pml:84, Error: assertion violated
spin: text of failed assertion: assert((avail>=0))
 12: proc 3 (Consumer:1) lockless.pml:84 (state 15) [assert((avail>=0))]
spin: trail ends after 12 steps
#processes: 4
        avail = -1
```

Questions?



https://spinroot.com